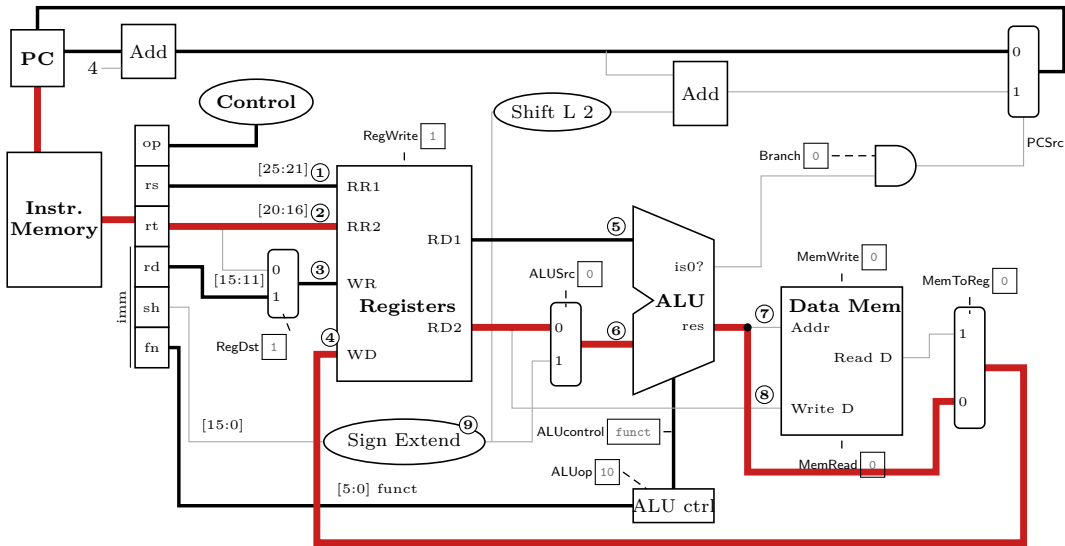


DATAPATH & CONTROL 1/4 single cycle, L11–12. **Thick red** = usual critical path; **bold** wire = used; grey wire = carries a value nobody keeps. Grey values = fixed for this type; blanks = yours.

C1 · R-type add sub and or slt \$rd, \$rs, \$rt

bold = the path this instruction uses



funct→ALUctrl: 20 add 0010, 22 sub 0110, 24 and 0000, 25 or 0001, 2A slt 0111. slt: WD = 1 if [rs] < [rt].
e.g. sub \$25, \$20, \$5 = 0285C822: \$20, \$5, \$25, [\$20]−[\$5], [\$20], [\$5], [\$20]−[\$5], [\$5], 0000C822 (Tut 4 Q3c)

1 split into fields

op	rs	rt	rd	shamt	funct
000000				00000	

2 the cells (U = kept; N = carried but nobody keeps it)

	cell	always	this question
①	RR1	rs	U
②	RR2	rt	U
③	WR	rd (RegDst = 1)	U
④	WD	ALU result = [rs] op [rt]	U
⑤	Opr1	[rs]	U
⑥	Opr2	[rt] (ALUSrc = 0)	U
⑦	Address	[rs] op [rt]	N
⑧	Write Data	[rt]	N
⑨	SignExt out	SignExt(rd shamt funct): ALU ignores it	N

3 control

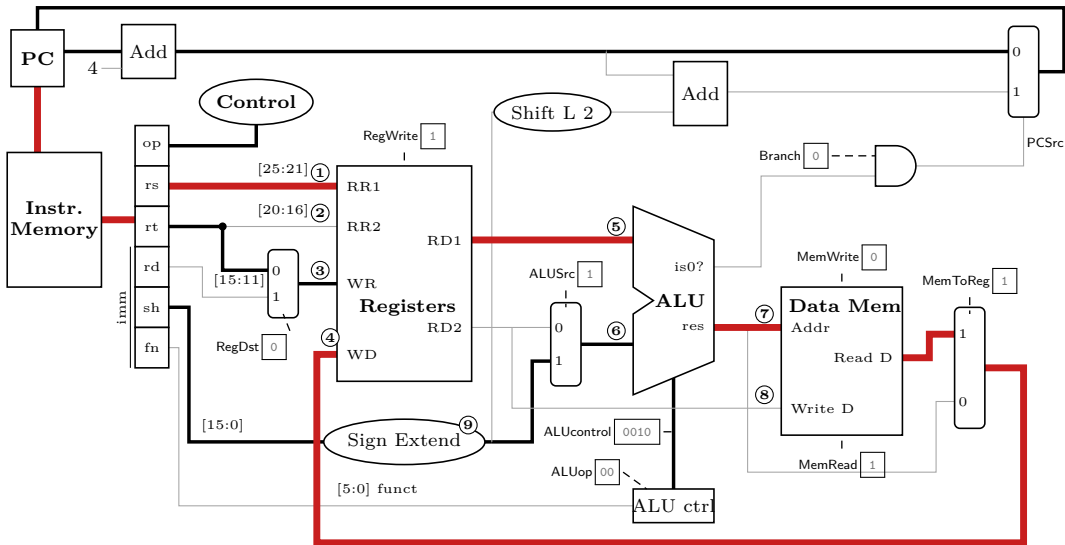
RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUop	ALUctrl
1	1	0	0	0	0	0	10	funct

4 next PC = PC + 4

5 critical path: IM → max(RF read → **ALUSrc mux**, Ctrl→ALUctrl) → ALU → MemToReg mux → RF write (bold = usually wins the max)

C2 · lw lw \$rt, imm(\$rs)

op 0x23 · rt is the destination



MemToReg mux is swapped: 1 (top) = memory. Sign-extended: −8 → FFFFFFF8. Slowest instruction (8 ns).
e.g. lw \$24, 0(\$15) = 8DF80000: \$15, \$24, \$24, Mem([\$15]+0), [\$15], 0, [\$15]+0, [\$24], 0 (Tut 4 Q3b)

1 split into fields

op	rs	rt	imm
100011			

2 the cells (U = kept; N = carried but nobody keeps it)

	cell	always	this question
①	RR1	rs	U
②	RR2	rt (read anyway, RD2 goes nowhere useful)	N
③	WR	rt (RegDst = 0)	U
④	WD	Mem([rs] + imm) (MemToReg = 1)	U
⑤	Opr1	[rs]	U
⑥	Opr2	SignExt(imm) (ALUSrc = 1)	U
⑦	Address	[rs] + imm	U
⑧	Write Data	[rt] (MemWrite = 0)	N
⑨	SignExt out	SignExt(imm) → ALUSrc 1	U

3 control

RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUop	ALUctrl
0	1	1	1	0	1	0	00	0010

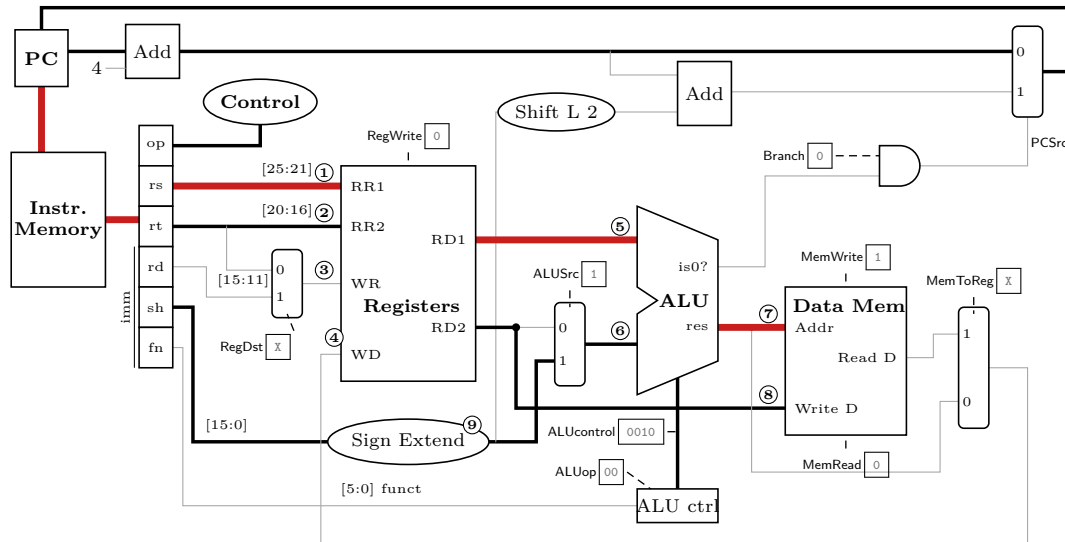
4 next PC = PC + 4

5 critical path: IM → max(RF read, Ctrl→**ALUctrl**, SignExt→ALUSrc mux) → ALU → DM → MemToReg mux → RF write (bold = usually wins; T5: tie at 600)

DATAPATH & CONTROL 2/4

C3 · sw sw \$rt, imm(\$rs)

op 0x2B · nothing is written to a register

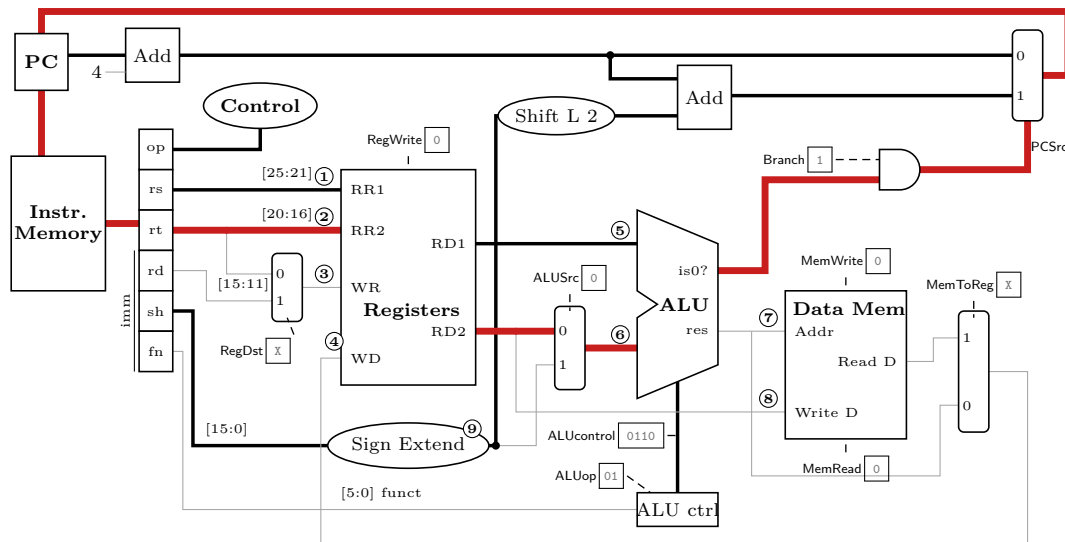


RegWrite = 0 $\Rightarrow$ RegDst, MemToReg = X: write *both* options for WR, WD. MemRead = 0 $\Rightarrow$ Read Data is random.

e.g. sw \$7, -8(\$29) = AFA7FF8: \$29, \$7, \$7 or \$31 (imm[15:11] = 1111), [\$29]-8 or random, [\$29], FFFFFFF8, [\$29]-8, [\$7], FFFFFFF8

C4 · beq	beq \$rs, \$rt, imm
----------	---------------------

op 0x04 · *imm counts instructions from PC+4*



ALU subtracts only to test equality; the result is thrown away. Shift L 2 = the $\times 4$.

e.g. beq \$1, \$3, 12 = 1023000C: \$1, \$3, \$3 or \$0, [\$1]–[\$3] or random, [\$1], [\$3], [\$1]–[\$3], [\$3], 0000000C; next PC = PC+4 or PC+4+48 (Tut 4 Q3a)

1 split into fields

op	rs	rt	imm
101011			

2 the cells ($\mathbf{U}$ = kept; $\mathbf{N}$ = carried but nobody keeps it)

	cell	always		this question
①	RR1	rs	U	
②	RR2	rt (the data to store)	U	
③	WR	rt or imm[15:11] (RegDst = X)	N	
④	WD	[rs]+imm or random (MemToReg = X)	N	
⑤	Opr1	[rs]	U	
⑥	Opr2	SignExt(imm) (ALUSrc = 1)	U	
⑦	Address	[rs] + imm	U	
⑧	Write Data	[rt] (MemWrite = 1)	U	
⑨	SignExt out	SignExt(imm) → ALUSrc 1	U	

3 control

RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUop	ALUctrl
X	0	1	0	1	X	0	00	0010

4 next PC = PC + 4

5 critical path: IM $\rightarrow$ max(RF read, Ctrl $\rightarrow$ ALUctrl, SignExt $\rightarrow$ ALUSrc mux) $\rightarrow$ ALU $\rightarrow$ DM write
(no MemToReg, no RF write)

1 split into fields

op	rs	rt	imm
000100			

2 the cells (**U** = kept; **N** = carried but nobody keeps it)

	cell	always		this question
①	RR1	rs	U	
②	RR2	rt	U	
③	WR	rt or imm[15:11] (RegDst = X)	N	
④	WD	[rs]—[rt] or random (X)	N	
⑤	Opr1	[rs]	U	
⑥	Opr2	[rt] (ALUSrc = 0)	U	
⑦	Address	[rs] − [rt] (only is0? is kept)	N	
⑧	Write Data	[rt]	N	
⑨	SignExt out	SignExt(imm) → Shift L 2 → Add	U	

3 control

RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUop	ALUctrl
X	0	0	0	0	X	1	01	0110

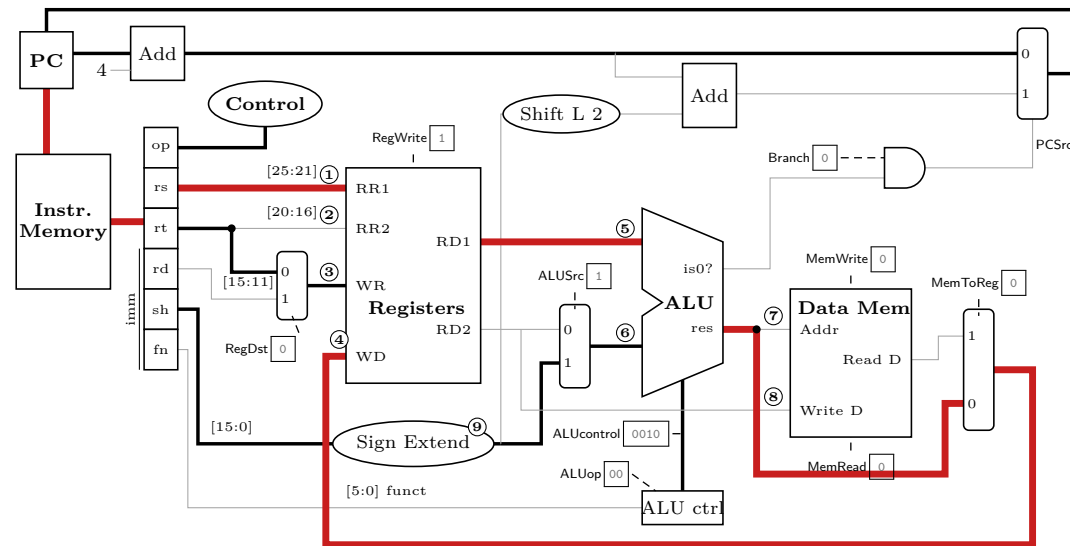
```
4 next PC = is0 = 1 ([rs] = [rt]): (PC+4) + imm×4 else PC+4. PCSrc = Branch AND
is0
```

5 critical path: IM $\rightarrow$ max(**RF read** $\rightarrow$ **ALUSrc mux**, Ctrl $\rightarrow$ ALUctrl) $\rightarrow$ ALU $\rightarrow$ AND $\rightarrow$ PCSrc mux (SignExt $\rightarrow$ Shift $\rightarrow$ Add runs in parallel)

DATAPATH & CONTROL 3/4 single cycle, L11-12. **Thick red = usual critical path**; **bold** wire = used; grey wire = carries a value nobody keeps. Grey values = fixed for this type; blanks = yours.

C5 · addi addi \$rt, \$rs, imm

op 0x08 . lw without the memory . AY25/26 Q28



vs lw: only MemRead 0 and MemToReg 0 differ. **andi/ori are NOT supported:** this datapath always sign-extends (L11); they need zero-extend + an AND/OR ALUop.

e.g. addi \$8, \$9, -3 = 2128FFFD: \$9, \$8, \$8, [\$9]-3, [\$9], FFFFFFFD, [\$9]-3, [\$8], FFFFFFFD

REF	the whole control unit on one card
-----	------------------------------------

	op	RegDst	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	ALUctrl
R-type	00	1	0	0	1	0	0	0	10	funct
lw	23	0	1	1	1	1	0	0	00	0010
sw	2B	X	1	X	0	0	1	0	00	0010
beq	04	X	0	X	0	0	0	1	01	0110
addi	08	0	1	0	1	0	0	0	00	0010

addi is not in the L12 table but is in L11's subset: it is lw without the memory. Midterms ask it.

ALUop	for	ALU does
00	lw, sw	add (address)
01	beq	subtract (compare)
10	R-type	look at funct

ALUctrl	
0000	AND
0001	OR
0010	add
0110	sub
0111	slt
1100	NOR

1 split into fields

op	rs	rt	imm
001000			

2 the cells (U = kept; N = carried but nobody keeps it)

	cell	always		this question
①	RR1	rs	U	
②	RR2	rt (read anyway, RD2 goes nowhere useful)	N	
③	WR	rt (RegDst = 0)	U	
④	WD	[rs] + imm (MemToReg = 0)	U	
⑤	Opr1	[rs]	U	
⑥	Opr2	SignExt(imm) (ALUSrc = 1)	U	
⑦	Address	[rs] + imm (DM off)	N	
⑧	Write Data	[rt] (MemWrite = 0)	N	
⑨	SignExt out	SignExt(imm) → ALUSrc 1	U	

3 control

RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	ALUop	ALUctrl
0	1	1	0	0	0	0	00	0010

$$4 \text{ next PC} = \text{PC} + 4$$

5 critical path: IM $\rightarrow$ max(**RF read**, **Ctrl** $\rightarrow$ **ALUctrl**, SignExt $\rightarrow$ ALUSrc mux) $\rightarrow$ ALU $\rightarrow$ MemToReg mux $\rightarrow$ RF write. T5: 400+200+120+30+200 = **950 ps**

Reading a signal

- RegDst: 0 = rt [20:16], 1 = rd [15:11]. ALUSrc: 0 = RD2, 1 = SignExt(imm).
- MemToReg: **1 = memory**, 0 = ALU result (mux drawn swapped).
- PCSrc = Branch AND is0: 1 = (PC+4) + SignExt(imm)×4.
- ALUctrl bits = Ainvert, Binvert, Operation(2). $ALUctrl_2 = ALUop_0 + ALUop_1 \cdot F_1$.

The cell rules

- Every wire carries *something* every cycle. Fill all 9 cells even when unused.
- Mux select X $\Rightarrow$ write both inputs: “A **or** B”.
- MemRead = 0 $\Rightarrow$ Read Data is *random*.
- X only appears where the result is thrown away (RegWrite = 0).
- Registers are read at the start and written at the clock edge: the same register as source and destination reads the *old* value.

Latency: add the blocks along the path; where paths run side by side take the *max*. Muxes and RF usually count twice (ALUSrc + MemToReg; read + write). L12: IM 2, RF 1, ALU 2, DM 2 ns $\Rightarrow$ R 6, lw 8, sw 7, beq 5, addi 6; clock = slowest. Formulas + ps version: card TIME.

Midterm facts: RR1/RR2/WR are 5 bits (32 regs). PC updates on the *rising* edge of the next cycle. MemToReg inputs are swapped only to keep the diagram tidy. ALU control: 2-bit ALUop + 6-bit funct in, 4 bits out. Drop the MemToReg mux $\Rightarrow$ lw breaks. To force is0 = 1, make the ALU result 0 (e.g. 0(\$zero)).

DATAPATH & CONTROL 4/4 single cycle, L11–12. **Thick red = usual critical path**; **bold** wire = used; grey wire = carries a value nobody keeps. Grey values = fixed for this type; blanks = yours.

TIME latency formulas, 1-bit ALU, control as code (Tut 5)

One formula per type (max = paths side by side; **bold** = usually wins. R/beq: the mux sits *after* RF so RF+MUX wins; lw/sw/addi: RF ties CTRL+ALUC, the SE mux path never wins)

R	IM + max(RF + MUX , CTRL + ALUC) + ALU + MUX + RF
lw	IM + max(RF , CTRL + ALUC , max(SE, CTRL) + MUX) + ALU + DM + MUX + RF
sw	IM + max(RF , CTRL + ALUC , max(SE, CTRL) + MUX) + ALU + DM
beq	max(IM + max(RF + MUX , CTRL + ALUC) + ALU + AND, IM + SE + SHL + ADD) + MUX
addi	IM + max(RF , CTRL + ALUC , max(SE, CTRL) + MUX) + ALU + MUX + RF

Shortcut when RF is the biggest parallel box: R = IM+RF+MUX+ALU+MUX+RF, lw = IM+RF+ALU+DM+MUX+RF, sw = IM+RF+ALU+DM, beq = IM+RF+MUX+ALU+AND+MUX. The first MUX in R/beq is ALUSrc (it waits for RD2); lw/sw/addi take SE, which is early.

	R	lw	sw	beq	addi	
L12 (ns)	6	8	7	5	6	clock 8 ns
Tut 5 (ps)	980	1300	1070	800	950	clock 1300 ps

L12: IM 2, RF 1, ALU 2, DM 2, rest 0. Tut 5: IM 400, Add 100, MUX 30, ALU 120, RF 200, DM 350, Ctrl/ALUC 100, SE/SHL/AND 20.

Q28-style ticks for addi: IM **on**; RF **on, twice**; MUX **on** (MemToReg only; ALUSrc mux done at 530 < RD1 at 600); ALU on; DM off; SE/shift off; Control ties with RF at 600 (key says off, say “tie”); **Adder off**: PC+4 starts at $t = 0$ beside IM and is done at 100, then just waits for the clock edge. Beq’s target adder: 440+100 = 540 < 800, off too.

Other instructions: what to add, and the new critical path (new parts: ZeroExt / gate / shift 20, extra mux 30; Tut 5 numbers in brackets)

instr	what to add	new critical-path formula	asked as
bne	signal BNE; PCSrc = Branch·is0 + BNE· $\overline{\text{is0}}$ (ALU same as beq, ALUop 01)	IM + max(RF + MUX , CTRL+ALUC) + ALU + AND + OR + MUX (820)	“needs new ALUcontrol” F (23/24 Q9)
j	ShiftL2 on inst[25:0], concat PC+4[31:28], Jump mux after PCSrc mux, signal Jump	IM + max(SHL, CTRL) + MUX (530): the mux waits for its <i>select</i>	no MemToReg mux needed (24/25 Q22); 256 MB block T (21s2 Q8)
jr	RD1 into another PC mux, signal JR	IM + max(RF , CTRL) + MUX (630)	–
jal	j hardware + RegDst input 31 + MemToReg input PC+4	IM + CTRL + MUX + RF (730) (write \$31; jump itself is 530)	–
sll / srl	shamt [10:6] into ALU + shifter; rs = 0 so RR1 unused	same as R: IM + max(RF + MUX , CTRL+ALUC) + ALU + MUX + RF (980)	“RR1 unused” T (23/24 Q10)
lui	imm $\ll$ 16 into a 3rd MemToReg input (skips the ALU); RR1 unused	IM + max(SHL16, CTRL) + MUX + RF (730)	same (23/24 Q10)
andi / ori	ZeroExt + ExtOp mux before ALUSrc mux; ALUop for AND / OR	IM + max(RF , CTRL + ALUC , max(ZE, CTRL)+MUX+MUX) + ALU + MUX + RF (950)	MemToReg flipped $\Rightarrow$ ori breaks (21s2 Q10)
slti	new ALUop code (11 is free)	same as addi (950)	–
nor	ALU-control row funct 27 $\rightarrow$ 1100	same as R (980)	–

1-bit ALU ALUctrl = Ainvert | Binvert | Operation(2): 00 AND, 01 OR, 10 adder, 11 less (slt)

op	Ainv	Binv	Oper	Cin (bit 0)	why
AND	0	0	00	–	
OR	0	0	01	–	
add	0	0	10	0	
sub	0	1	10	1	$A + \overline{B} + 1 = A - B$
slt	0	1	11	1	subtract, take the sign bit
NOR	1	1	00	–	$\overline{A} \cdot \overline{B}$ (De Morgan)

Cin only enters the adder. Cin(bit 0) = Binvert. Overflow = Cin(MSB) XOR Cout(MSB): pos+pos $\rightarrow$ neg or neg+neg $\rightarrow$ pos.

Control from the opcode (Tut 5 Q5; + addi where noted)
R = (op==0x00); lw = (op==0x23); sw = (op==0x2B); beq = (op==0x04)
RegDst = R; ALUSrc = lw|sw|addi; MemToReg = lw
RegWrite = R|lw|addi; MemRead = lw; MemWrite = sw
Branch = beq; ALUop = (R<1) | beq // 10, 01, 00

ALU control / ALU / MUX (Tut 5 Q4, Q6, Q7)
if (ALUop==0) return 0x2; if (ALUop==1) return 0x6;
switch (funct) { 0x20: 0x2; 0x22: 0x6; 0x24: 0x0; 0x25: 0x1; 0x2A: 0x7 }
ALU: res = op(in0,in1); *ALUiszero = (res == 0); return res;
#define MUX(N) uint###N##_t mux##N(bool s, uint###N##_t in0, uint###N##_t in1) { return s ? in1 : in0; }